Why Quantum? A Staged, Evidence-Based Case for Quantum-Ready Route-Pool Optimization in VRPTW

HARP-QUBO Project
Internal technical note

September 6, 2026

Abstract

This document lays out, step by step, the reasoning and empirical evidence behind our project's motivation for formulating the VRPTW route-selection problem as a QUBO and for pursuing quantum computing (QAOA / quantum annealing) as a long-term solution direction. It is written to stand on its own: every claim is either backed by a real experiment run within this project (with the exact numbers reported, not rounded impressions) or by a cited, verifiable reference. Where our evidence is incomplete, ambiguous, or points the other way, that is stated plainly rather than smoothed over – several steps below required us to revise or discard an earlier, weaker version of the argument. The storyline has eight parts, corresponding to a slide/argument sequence discussed internally; each part here includes the supporting literature and/or the supporting data.

Contents

1 Why VRPTW matters

The Vehicle Routing Problem with Time Windows (VRPTW) asks for a set of vehicle routes, each starting and ending at a depot, that together serve every customer exactly once, respect vehicle capacity, and respect each customer's allowed delivery time window, while minimizing total cost (typically distance and/or lateness). It is one of the most heavily studied NP-hard combinatorial optimization problems because it directly models last-mile delivery, field service scheduling, and freight distribution – industries where even a 1–2% routing efficiency gain translates into large real-world cost and emissions savings. The problem's canonical benchmark instances were introduced by Solomon [1], and remain the standard testbed used throughout this work (the Solomon "R"-series 100-customer instances).

Because VRPTW is NP-hard, no known classical algorithm solves large instances exactly in polynomial time; practical solvers rely on heuristics and metaheuristics that trade a certificate of optimality for tractable runtime. This trade-off is the starting point for everything that follows.

2 Multi-start pooling of metaheuristic solutions is standard, established practice

Our pipeline's core architecture is: run a metaheuristic (Large Neighborhood Search, LNS) many times from different random seeds/starting points, pool the routes produced across all of those runs, and then solve a set-partitioning selection problem over that pool to assemble a final solution. This is not an ad-hoc idea; it is a well-established family of methods in the operations research literature:

- **Multi-start metaheuristics.** Martí, Resende & Ribeiro [3] survey multi-start methods for combinatorial optimization broadly: strategically sampling the solution space via repeated, independently-seeded search runs, then combining or selecting among the results, is shown across many problem classes to outperform any single run.
- **Large Neighborhood Search (LNS).** Pisinger & Røpke [4] survey LNS and its adaptive variants, which is the specific metaheuristic engine used in our pipeline's destroy-and-repair loop.
- **Pool-and-select for vehicle routing specifically.** Rochat & Taillard [2] is the closest and most important prior art to our architecture: they show that running local search repeatedly with controlled randomization, then *recombining routes drawn from different runs* via a matheuristic (set-partitioning) step, systematically improves on the best single run for

vehicle routing problems – improving on the literature's best-known solutions for nearly forty benchmark instances at the time. Our project's "build many LNS pools → pool their routes → select via QUBO" architecture is a direct descendant of this idea.

Honest caveat

This citation (Rochat & Taillard, 1995) is the single most relevant piece of prior art to our whole architecture, and as of this writing it is still *not yet added* to `paper/overleaf/sections/02_related_work.tex` in the manuscript. This is a known, flagged gap that must be closed before submission – a reviewer working in this subfield would very likely know this paper and would flag its absence.

3 Does combining routes from multiple independent LNS runs actually beat a single run? – Yes, verified directly

Rather than assume the literature's finding transfers to our exact pipeline and QUBO encoding, we tested it directly on a real Solomon instance (r101, 100 customers reduced/kept, $K = 8$ vehicles).

Method. We ran LNS independently from 8 different random seeds (601–608), each exploring 150 independently-built starting solutions with 120 destroy-and-repair iterations per start (18,000 iterations per seed, 144,000 total), and pooled every distinct route surfacing from any of these runs together with an existing 600-route baseline pool (itself already the product of an earlier, smaller 3-seed LNS campaign). We then solved the resulting route-selection problem *exactly* (branch-and-bound, not a heuristic) at increasing truncations of this pool.

Established finding

At $N = 600$ routes drawn from the genuinely multi-seed pool, the exact solver certified an optimal solution of score **22,413.19**. This is better than the best score found by *any other pool tested in this project* on the same instance, including the original 600-route single-campaign pool and a much larger 1,673-route pool built with additional plain construction-heuristic routes (both of which converged to a best of 22,430.53). Combining routes from multiple independent LNS runs found a genuinely better solution than any single pool contained – a direct, positive confirmation of the Rochat–Taillard-style hypothesis on our own pipeline and instance.

Honest caveat

This is currently a *single* verified instance (r101_keep100), not a statistically characterized result across multiple instances/seeds. A rigorous version of this claim for publication should report how often, and by how much on average, multi-seed pooling beats single-run pooling across a representative set of instances – not one favorable anecdote.

4 Full-dataset validation (90 cases): where the approach loses, and whether more LNS exploration can rescue it

Section 3's finding was a single instance. We subsequently ran the identical multi-seed-pooling pipeline across the complete Solomon 100-customer dataset (45 instances $\times$ keep$\in \{70, 100\}$ = 90 cases: 24 R-family, 34 C-family, 32 RC-family), 8 seeds per case, comparing the exact-solved pool against an independent OR-Tools benchmark (30-second time limit, never inserted into the candidate pool).

Family	n	Mean exact time	`neal` valid	Swap valid	Beat	Tied	Lost
C	34	0.0015 s	57.7%	99.7%	13	19	2
R	24	83.71 s*	0.05%	45.1%	21	0	1
RC	32	13.98 s	1.73%	73.3%	20	0	12

Table 1: Full 90-case sweep, family-wise. *Of 22 attempted; 2 R-family cases exceeded the 700-route exact-solve cutoff and were skipped. "Beat"/"Tied"/"Lost" are against the OR-Tools benchmark.

Established finding

R-family's wins are all genuine (21 strict beats, zero ties): when multi-seed pooling wins against OR-Tools here, it is a real, distinct improvement, not a coincidental score match. C-family's high beat-or-tied count is mostly *ties* (19 of 32): both methods frequently converge on the same answer, consistent with the pool-collapse pattern below, rather than our approach outperforming OR-Tools. RC-family is the one genuinely contested family: 20 real wins and 12 real losses, no ties at all.

Honest caveat

C-family pools frequently collapse to near the minimum possible size. A representative case, c106_keep70 ($K = 7$), pooled routes from 8 independent LNS seeds and ended with a final pool of exactly **7 unique routes** – i.e., every seed converged on the same single candidate cover, contributing zero additional diversity beyond one seed's output. We reproduced this independently outside the main sweep (identical settings, same result), so it is a property of the instance and destroy/repair operators, not a one-off artifact. With only K routes available, there is exactly one way to select K of them, so exact/`neal`/swap-annealer are not actually solving a selection problem at all in these cases – this is the direct explanation for C-family's near-zero mean exact time (1.5 ms) and unusually high `neal`/swap validity (both methods trivially succeed when there is effectively nothing to select among).

4.1 Where the losses come from: a truncation risk, and a route-generation limitation

Every one of the 15 losses was inspected individually (instance, keep level, exact score, OR-Tools score, and pool size before/after the family-dependent master-pool cap). Two distinct, separable causes emerged:

Cause 1 – aggressive truncation of an unusually large raw pool. The single largest loss by far, rc206_keep70, lost by **167%** (score 6,176.29 vs. OR-Tools' 2,310.34) – an order of magnitude

worse than every other loss (which ranged 0.6%–17.7%). rc206's multi-seed campaign found 2,100 unique routes before the RC-family's fixed 600-route master-pool cap truncated it down to the 600 individually-cheapest routes, discarding 71% of what was found. rc201_keep70 (2,150 routes before cap) shows the same pattern, though with a milder 8.58% loss. Truncating by *individual* route score, with no regard for which combinations survive together, can and apparently does discard routes that were structurally necessary for a good combined solution – exactly the mechanism identified in Section 5's honest caveat, now observed causing real losses, not just slower solves.

Cause 2 – tested directly: does more LNS exploration rescue the remaining losses? For two RC-family losses with *small, uncapped* pools (so Cause 1 does not apply), we tested whether additional LNS search effort – more seeds, and separately, seeds run with substantially more aggressive destroy/repair parameters (`insertion_noise`: $0 \to 0.2$; `remove_counts`: $\{3,4,5,6,8\} \to \{10,15,20,25\}$, i.e. larger, more disruptive perturbations per iteration) – could close the gap to OR-Tools.

Case	Baseline (pool, exact)	OR-Tools	After more exploration	Outcome
rc101_keep70	90 routes, 5,539.30	5,282.69	389 routes, 5,535.15	Still loses (by 252.45)
rc103_keep70	140 routes, 2,003.23	1,884.83	507 routes, **1,832.92**	**Now wins (by 51.91)**

Table 2: Effect of additional, more aggressive LNS exploration on two RC-family losses (same construction seed, same instance; only the LNS destroy/repair budget and parameters differ from the original 8-seed campaign).

Honest caveat

The result is genuinely mixed, and that is itself informative. For rc103_keep70, one additional aggressively-parameterized seed grew the pool from 140 to 394 routes and immediately unlocked a combination 170.31 points better than the original best – turning a 118.39-point loss into a 51.91-point win. For rc101_keep70, seven additional standard-parameter seeds (pool grown to 178, zero score movement across every single one) followed by an aggressive-exploration seed (pool grown further to 389) produced only a 4.15-point improvement – nowhere near closing a 256.61-point gap. This is not a uniform "just search more" fix: for at least some instances (rc103), the original 8-seed campaign had simply not explored enough of the reachable route space yet, and more LNS effort resolves it directly; for others (rc101), the gap appears to reflect a more structural limitation in what this destroy-and-repair method can produce for that instance, not merely an insufficiently large search budget. Distinguishing these two cases in advance, rather than only after the fact, is an open problem this document does not resolve.

A second, independently useful observation from this same follow-up: on rc103's newly-extended 507-route pool – the one where a winning combination now demonstrably exists – `neal` still found it in only 1 of 1,000 samples (best found: 2,609.04, worse than both exact *and* OR-Tools) and the swap-move annealer in 129 of 200 restarts but never the winning combination itself (best found: 2,057.07, again worse than both). **Even when a demonstrably better solution is confirmed to exist in the pool, neither classical heuristic reliably finds it** – only exhaustive search does. This is the same capability gap from Sections 7–8, now reproduced on a third, independent instance.

5 The cost of that improvement: pooled routes make the selection problem too large to solve exactly

The Rochat–Taillard-style benefit above did not come for free. We measured the real (not extrapolated) runtime of the exact branch-and-bound selector as the pool size N grows, on the same genuinely multi-seed-LNS-augmented pool from Section 3.

N (routes)	Time	Feasible	Best certified score
200	25.0 sec	No	–
300	11.5 min	No	–
400	43.9 min	Yes	23,100.75
500	2.10 hr	Yes	22,965.55
600	5.23 hr	Yes	**22,413.19**
700	9.59 hr	Yes	22,413.19 (unchanged)

Table 3: Real, measured exact-solver runtime on the genuinely multi-seed-LNS pool, instance r101_keep100, $K = 8$.

An exponential fit to this data ($\log(\text{time}) = 4.53 + 0.0086N$, $R^2 = 0.986$ on the feasible points) extrapolates to roughly **8.8 days** to exactly solve the full 1,045-route pool this campaign produced. Time did not grow smoothly: the per-100-route multiplier itself decreased across the range (27.6×, 3.83×, 2.87×, 2.49×, 1.83×), so this extrapolation is best read as an order-of-magnitude upper indication, not a precise prediction – but the direction and scale of the wall is unambiguous.

Honest caveat

This result is specific to route *competitiveness*, not merely route *count*. An earlier, separate test on this project extended the pool instead with routes from a plain construction heuristic (no LNS) rather than genuine independently-seeded LNS output, reaching 1,673 routes. That pool's exact-solve time *flattened out* – 535 seconds at $N = 1{,}400$, not days. The mechanism: plain construction-stage routes are mostly mediocre and get pruned by branch-and-bound almost for free; genuinely LNS-refined routes are mutually competitive (many near-tied alternatives), which is exactly what makes branch-and-bound's pruning ineffective. This means the paper's claim must be stated precisely: *it is not pool size per se that breaks exact solving – it is the accumulation of many genuinely competitive alternatives, which is precisely what makes a pool useful in the first place (Section 3).* The same property that makes multi-seed LNS pooling valuable is the property that makes exact solving intractable. Separately, across two independent LNS campaigns (3 seeds, then 8 more seeds, 144,000+ total destroy/repair iterations), the pool saturated at only 1,045 unique routes – far below a naive expectation that more seeds keep adding proportionally more routes. Route diversity across independent LNS seeds is itself limited; this is consistent with, and reinforces, an earlier finding in this project's investigation into LNS route-pool diversity.

6 Formulating route selection as a QUBO: a quantum-ready encoding

Given a pool of N candidate routes and a requirement to select exactly K of them that jointly cover every customer exactly once at minimum total cost, we encode the selection as a Quadratic Unconstrained Binary Optimization (QUBO) problem: one binary variable $y_i \in \{0, 1\}$ per candidate route, a linear cost term per selected route, a quadratic penalty term enforcing that each customer is covered by exactly one selected route, and a quadratic penalty enforcing that exactly K routes are selected.

This formulation is deliberately chosen because a QUBO is the native input format for quantum annealers (e.g. D-Wave-style hardware) and maps directly onto the cost Hamiltonian used by gate-model quantum algorithms such as QAOA. Prior work has already demonstrated this style of formulation for vehicle routing on real quantum annealing hardware: Feld et al. [5] solve the capacitated VRP's routing phase as a QUBO on a D-Wave quantum annealer, evaluating whether the quantum-classical hybrid approach is competitive with classical baselines. Our project positions the QUBO route-selection layer analogously – as a formulation whose classical solve is used today (via simulated annealing / exact search), but which is structurally ready to be handed to quantum hardware as it matures, without reformulation.

7 Where the classical QUBO solver breaks: the bit-flip / `neal` simulated-annealing problem

Our production pipeline solves the QUBO above using `neal`'s classical simulated-annealing sampler, which explores the solution space via single-bit-flip moves (flip one y_i at a time). We measured this sampler's actual validity rate – the fraction of its 1,000 samples per run that satisfy both constraints (exactly K routes, full non-overlapping coverage) – across every real case this project has run at production scale (600-route pools, all available Solomon R-series instances, both `keep70` and `keep100` settings), using the pipeline's own saved historical output (no re-run needed):

Statistic	Valid samples / 1000	Validity rate
Mean (18 cases)	6.28	0.63%
Maximum (r105_keep70)	20	2.0%
Minimum (five separate cases)	1	0.1%

Table 4: Real, historical QUBO+`neal` validity rate at 600-route pool scale, across all available real instances.

Why this happens. The number of quadratic terms in this QUBO is dominated by the exactly-K cardinality penalty, which couples every pair of routes: $\binom{N}{2} + N$ terms. At $N = 600$ that is exactly 180,300 terms (confirmed directly from the pipeline's own logged `qubo_terms` field across all 18 cases above). This dense coupling means a single-bit-flip search has to find its way to one of a vanishingly small number of exactly-K, fully-covering states inside an search graph whose edge density explodes combinatorially with N ($\binom{N}{2}$: 66 edges at $N = 12$, versus 179,700 at $N = 600$), while the sampler's search budget (number of reads/sweeps) stays fixed. The same algorithm, run at $N = 12$ on a small verified toy problem within this project, found valid solutions in 1,809 of 2,000 samples (90.5%) – confirming the mechanism is pool scale, not a flaw in `neal` itself or in the

penalty-weight tuning (both were explicitly ruled out as the cause during this investigation).

8 A classical fix: restructuring the search with cardinality-preserving swap moves

If single-bit-flip moves are the problem, the natural classical fix is to search only over moves that automatically preserve the exactly-K constraint: a *swap* move that removes one currently-selected route and adds one currently-unselected route, always keeping exactly K routes selected. We implemented this as a Metropolis–Hastings annealer (geometric cooling, $T_0 = 3.0 \to T_1 = 0.005$) minimizing coverage_weight $\times$ coverage_error + score_sum/1000, where coverage_error counts customers not covered exactly once.

This swap-move restructuring was validated earlier in this project across all 24 real R-series cases and found to reliably restore validity where the bit-flip QUBO+`neal` approach failed outright, though solution *quality* (how close to the true optimum) remained inconsistent. Re-testing on the genuinely multi-seed-LNS pool from Section 5 confirms the same qualitative pattern:

N	Time (20 restarts)	Valid restarts	Best score found
200	1.68 sec	0/20	none valid
400	1.78 sec	1/20	23,100.75
600	1.82 sec	6/20	23,331.14
800	1.78 sec	10/20	23,335.57
1000	2.15 sec	4/20	23,182.42
1045 (full pool)	1.89 sec	3/20	23,579.69

Table 5: Swap-move annealer: runtime stays flat regardless of N (contrast Table 3), but quality never approaches the exact optimum and reliability is inconsistent.

9 Toward a fair test of quantum advantage: the XY-mixer QAOA proposal

9.1 What the data actually shows – stated precisely

It is tempting to read Tables 3 and 5 together as: "the exact solver is too slow, the fast heuristic is unreliable and $\sim$3% worse than optimal (23,100.75 vs. 22,413.19) – therefore quantum hardware will solve this." **This is not a claim our evidence supports, and we deliberately do not make it.** What the data *does* support is narrower and more defensible: *no classical method we have tested is simultaneously fast and reliably optimal on this class of problem.* That specific kind of gap – a landscape with many competitive local optima that trap a fast local search, while an exhaustive search is prohibitively slow – is exactly the kind of structure for which quantum tunneling has been proposed, and in at least one crafted case empirically shown, to offer an advantage over classical simulated annealing (Denchev et al. [9], showing up to an 8$\times$ time-to-solution advantage for a D-Wave 2X annealer over single-core simulated annealing on a problem engineered with tall, narrow energy barriers). QAOA, the gate-model analogue explored in this project, was introduced by Farhi, Goldstone & Gutmann [6].

9.2 Why our own quantum-style test so far has *not* supported an advantage – and the specific, known reason

We tested a QAOA circuit (exact statevector simulation, not real hardware, but a faithful ideal-quantum-behavior model) against `neal` on a small, exhaustively-verified 12-route selection toy engineered to contain a genuine local-optimum barrier. The result: `neal` found a valid selection in 1,809 of 2,000 shots (90.5%); the QAOA circuit found a valid selection in only 7 of 2,000 shots (0.35%), and even those 7 landed on the *worse* of the two covers, never the better one.

Honest caveat

Taken at face value, this result argues *against* a quantum advantage, not for one. But the specific circuit tested used a standard *X-mixer* – the original QAOA mixer from Farhi et al. [6] – which does not preserve the "exactly K routes selected" constraint. An X-mixer drives probability mass across the *entire* 2^N-dimensional space, including the overwhelming majority of states that violate cardinality, so most of the circuit's probability mass is wasted on infeasible outcomes before validity is even checked. This is not a property of quantum computation in general; it is a specific, known, and fixable mismatch between the mixer and the constraint structure of this problem.

The correct fix, well established in the QAOA literature under the name *Quantum Alternating Operator Ansatz*, is to replace the X-mixer with a constraint-preserving *XY-mixer*: a mixing Hamiltonian built from $XX+YY$ interactions between qubits, which only ever moves probability amplitude between states of *equal Hamming weight* – i.e., it never leaves the "exactly K selected" subspace in the first place. This was introduced by Hadfield et al. [7] and analyzed in further detail by Wang et al. [8], including explicit application to exactly this kind of "choose K of N" combinatorial structure.

9.3 The proposed next experiment (not yet run)

We propose – and have not yet executed – rerunning the same 12-route trap toy (and, if informative, the swap-move annealer's own N-sweep in Table 5) with a properly constructed XY-mixer QAOA circuit, initialized in a uniform superposition over only the exactly-K-selected subspace, and compared against `neal` and the swap-move annealer under identical, fair, shot-based evaluation (same number of samples, same validity/quality metrics). This is the experiment that would actually test whether cardinality-preserving quantum search can close the gap that neither classical method in this project has closed – rather than asserting that it would.

Honest caveat

Until that experiment is run, any claim that "real quantum hardware would find the best solution" remains a hypothesis grounded in relevant theory and prior empirical precedent (Denchev et al.), not a demonstrated result of this project. We recommend this framing be used explicitly in any manuscript text derived from this document: the case for quantum readiness rests on (a) a demonstrated classical capability gap (Sections 5–8), and (b) a specific, theoretically well-motivated, not-yet-executed test of whether a properly constructed quantum algorithm can close it – not on an already-demonstrated quantum advantage.

10 Consolidated summary of all findings

Claim	Status and evidence
Multi-start pooling is standard OR practice	Confirmed via literature [3, 4, 2].
Multi-seed LNS pooling beats single-run pooling	Confirmed on r101_keep100: 22,413.19 vs. 22,430.53 (Sec. 3). Single-instance result; needs multi-instance statistics for publication.
Larger, more competitive pools defeat exact solving	Confirmed: 25 sec → 9.59 hr, N=200→700, on genuinely LNS-competitive pool (Sec. 5, Table 3). Driver is route *competitiveness*, not raw pool size – a construction-filler pool of similar/larger size did *not* show this wall.
QUBO is a sound, quantum-ready formulation	Standard practice, precedented on real quantum annealer hardware [5] (Sec. 6).
Bit-flip `neal` sampler fails at scale	Confirmed across all 18 available real production cases: mean 0.63% validity, mechanistically explained by $\binom{N}{2}$ + N cardinality-penalty coupling (Sec. 7, Table 4).
Swap-move restructuring restores validity	Confirmed across all 24 real cases (prior work) and reconfirmed on the new pool (Sec. 8, Table 5); quality remains ∼3% below the exact optimum and inconsistent across restarts.
Quantum (QAOA) already shown to help	**Not established.** Our only quantum-style test (X-mixer QAOA) underperformed `neal` badly (0.35% vs. 90.5% valid), for a specific, fixable reason (Sec. 9). The fair test (XY-mixer QAOA) has not yet been run.

11 Limitations and honest caveats

- **Single primary instance.** Sections 3 and 5's headline numbers come from one Solomon instance (r101, 100 customers reduced to `keep100`, $K = 8$). Section 7's validity table is broader (18 real cases), but the newer scaling and multi-seed-pooling results are not yet replicated across instances.
- **Pool growth saturates.** Despite two independent LNS campaigns totaling 11 seeds and well over 100,000 destroy/repair iterations, the genuinely-LNS-refined pool topped out at 1,045 unique routes. Claims about pool size should not assume unbounded growth with more computational effort.
- **No statistically confirmed broad multimodality.** An earlier hypothesis in this project – that the route-selection landscape is generically rugged/multimodal across all cases, independently motivating a tunneling-based quantum advantage – was tested with Hartigan's dip test across all 24 real cases and found *not* statistically significant in any of them (0/24). An earlier internal visualization (this project's "Two Webs" artifact) asserted rugged multimodality based on a smaller, 30-restart sample; that specific claim did not survive the larger, more rigorous re-test and the visualization has not yet been corrected to reflect this. This does not invalidate the arguments in this document (which rest on measured runtime and validity data, not on an assumed landscape shape), but it does mean the tunneling argument

in Section 9 should be framed via the demonstrated capability gap and cited theory, not via an in-house claim of general landscape ruggedness.

- **Extrapolation uncertainty.** The exponential fit underlying the "8.8 days at full pool size" estimate in Section 5 has $R^2 = 0.986$ on the points used, but the per-step growth multiplier itself was observed to decelerate across the measured range, so the true asymptotic behavior beyond $N = 700$ is not fully characterized.

12 Recommended next steps

1. Run the fair XY-mixer QAOA experiment proposed in Section 9 before making any claim about quantum advantage in a manuscript.
2. Repeat the multi-seed-pooling-vs-single-run comparison (Section 3) and the exact-solver scaling measurement (Section 5) across multiple Solomon instances to replace single-instance anecdotes with statistics.
3. Add the Rochat & Taillard [2] citation to `02_related_work.tex`.
4. Measure the swap-move annealer's own scaling behavior with a larger restart budget to check whether its quality gap versus the exact optimum narrows with more search effort, or is a structural ceiling of the swap-move neighborhood itself.
5. Correct or annotate the "Two Webs" visualization to reflect the dip-test finding (Section 11), so internal materials do not contradict each other.

References

[1] M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.

[2] Y. Rochat and É. D. Taillard. Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics*, 1(1):147–167, 1995.

[3] R. Martí, M. G. C. Resende, and C. C. Ribeiro. Multi-start methods for combinatorial optimization. *European Journal of Operational Research*, 226(1):1–8, 2013.

[4] D. Pisinger and S. Røpke. Large Neighborhood Search. In M. Gendreau and J.-Y. Potvin, editors, *Handbook of Metaheuristics*, 2nd edition, pages 399–420. Springer, 2010.

[5] S. Feld, C. Roch, T. Gabor, C. Seidel, F. Neukart, I. Galter, W. Mauerer, and C. Linnhoff-Popien. A hybrid solution method for the capacitated vehicle routing problem using a quantum annealer. *Frontiers in ICT*, 6:13, 2019.

[6] E. Farhi, J. Goldstone, and S. Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[7] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas. From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms*, 12(2):34, 2019.

[8] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel. XY mixers: Analytical and numerical results for the quantum alternating operator ansatz. *Physical Review A*, 101:012320, 2020.

[9] V. S. Denchev, S. Boixo, S. V. Isakov, N. Ding, R. Babbush, V. Smelyanskiy, J. Martinis, and H. Neven. What is the computational value of finite-range tunneling? *Physical Review X*, 6:031015, 2016.